DOCUMENTAÇÃO TÉCNICA

MEC System.

Sistema Integrado de Atendimento
e Execução de Serviços

OFICINA MECÂNICA

Da necessidade à implementação.

Contexto e tecnologias

Arquitetura do projeto

Regras de negócio

Documentação em docs/apresentacao/

Por onde vamos?

01 Introdução

02 Stack utilizada

03 Arquitetura do projeto

04 Regras de negócio

Introdução

Contexto e motivo da implementação

Uma oficina mecânica de médio porte, especializada na manutenção de veículos, enfrenta dificuldades para expandir seus serviços mantendo qualidade e eficiência. O atendimento, o diagnóstico, a execução dos serviços e a entrega dos veículos dependem de anotações manuais e planilhas dispersas, sem um fluxo integrado de informações.

Essa organização provoca erros na priorização dos atendimentos, falhas no controle de peças e insumos, dificuldade para acompanhar o status dos serviços, perda de histórico de clientes e veículos e ineficiência na elaboração de orçamentos e obtenção de autorizações. À medida que a demanda cresce, a equipe precisa conferir informações em diferentes lugares, aumentando o retrabalho e dificultando a comunicação com o cliente.

Para enfrentar esses problemas, a oficina decidiu investir em um **Sistema Integrado de Atendimento e Execução de Serviços**, o MEC System. O projeto visa atender às demandas mencionadas por meio da centralização dos cadastros, dos orçamentos, do estoque e das ordens de serviço, criando uma base para uma gestão interna eficiente e segura.

A visão do produto inclui permitir que os clientes acompanhem em tempo real o andamento dos serviços e autorizem reparos adicionais por aplicativo. Esses objetivos orientam a evolução do sistema; a implementação atual concentra-se no backend e ainda não entrega toda essa experiência ao cliente.

Relação entre os problemas e o projeto

Demanda da oficina	Contribuição da implementação atual	Evolução necessária para o objetivo completo
Priorizar os atendimentos	Ordens registram status e datas; jobs identificam prazos próximos e atrasos.	Definir critérios e uma fila de priorização; não há algoritmo de prioridade implementado.
Controlar peças e insumos	Estoque registra quantidade, preço, lote e prazo de reposição. A criação de orçamento consulta os itens e acrescenta prazo quando a quantidade é insuficiente.	Completar o fluxo de movimentação, reserva e baixa de estoque associado à execução.
Acompanhar o serviço	A API permite consultar e atualizar ordens com status de recebimento, diagnóstico, espera de aprovação, execução, finalização e entrega.	Desenvolver a interface do cliente e a atualização em tempo real.
Preservar dados de clientes e veículos	Usuários, veículos, orçamentos e ordens possuem registros persistentes e relacionamentos no banco.	Complementar a consulta de histórico e, se necessário, auditoria das alterações; os registros atuais não constituem um histórico completo de eventos.
Organizar orçamentos e autorizações	Orçamentos vinculam cliente, técnico e veículo; ordens se vinculam a orçamentos e incluem o status `aguardando_aprovacao`.	Implementar o ato de aprovação/rejeição pelo cliente e o registro de consentimento para reparos adicionais.

Stack utilizada

Python

O que é e qual sua função: Python é a linguagem utilizada para implementar a API, as regras de negócio, a persistência, os jobs e os scripts. O pyproject.toml declara Python `>=3.12`, e a imagem Docker utiliza Python 3.12. A linguagem oferece sintaxe legível e recursos para programação assíncrona, empregados nas operações de banco e HTTP. Documentação do Python.

Justificativa: a escolha considera seu uso amplo no mercado, a documentação robusta e o ecossistema de bibliotecas. Esses fatores facilitam a formação da equipe, a manutenção e a evolução das regras de atendimento, sem exigir ferramentas próprias para tarefas comuns de backend.

SQL e PostgreSQL

O que são e qual sua função: SQL é a linguagem de consulta e manipulação de bancos relacionais; PostgreSQL é o sistema gerenciador adotado. Ele armazena usuários, veículos, orçamentos, estoque, ordens e notificações. O Compose utiliza PostgreSQL 16, com volume persistente. Os modelos usam chaves estrangeiras, identificadores únicos e uma coluna JSONB para os itens do orçamento. Documentação do PostgreSQL 16.

Justificativa: a escolha combina praticidade, documentação extensa e uso consolidado no mercado. O modelo relacional se ajusta às relações entre cliente, veículo, orçamento e ordem, enquanto transações e restrições ajudam a preservar consistência. O JSONB permite armazenar a lista de itens de um orçamento, com as limitações de integridade explicadas no MER.

FastAPI

O que é e qual sua função: framework web que organiza rotas HTTP, dependências, validação e documentação OpenAPI. Em `main.py`, reúne os routers, configura o ciclo de vida do cliente HTTP e registra os tratamentos globais de erros. Expõe Swagger em `/docs` e ReDoc em `/redoc`. Documentação do FastAPI.

Justificativa: a documentação extensa e a integração com tipagem, Pydantic e funções assíncronas simplificam a construção e manutenção da API. A documentação interativa facilita entender e experimentar os contratos consumidos por futuras interfaces.

Arquitetura do projeto

Organização arquitetural

O MEC System é um **monólito modular**, organizado por responsabilidades de negócio e por camadas. Os módulos `users`, `quotes` e `orders` compartilham uma aplicação FastAPI, uma base de código e um banco. O scheduler executa separadamente, usando a mesma imagem e os mesmos modelos.

Estrutura do backend

```
src/app/
├── main.py
├── config.py
├── bootstrap/
├── modules/
│   ├── users/
│   │   ├── domain/{entities,enums,value_objects}/
│   │   ├── application/{dtos,ports,services,use_cases}/
│   │   └── infrastructure/{clients,http,models,repositories}/
│   ├── quotes/
│   │   ├── domain/entities/
│   │   ├── application/{dtos,ports,use_cases}/
│   │   ├── infrastructure/{http,models,repositories}/
│   │   └── exceptions.py
│   └── orders/
│       ├── domain/{entities,enums}/
│       ├── application/{dtos,ports,use_cases}/
│       └── infrastructure/{http,jobs,models,repositories}/
└── shared/infrastructure/
    ├── database/
    ├── logging/
    └── security/
```

Camadas e responsabilidades

Pasta	Função no projeto
`domain/`	Reúne as estruturas e regras que representam os conceitos centrais da aplicação, separadas dos detalhes de transporte HTTP e
`application/`	Organiza as operações oferecidas pelo sistema e coordena a colaboração entre regras, contratos e serviços.
`infrastructure/`	Reúne as implementações técnicas que conectam a aplicação ao banco de dados, à interface HTTP e aos serviços externos.
`bootstrap/`	conecta implementações e casos de uso em arquivos como `users.py`, `budgets.py` e `orders.py`. Também contém comandos de criação de tabelas, migração de tipos de notificação, importação JSON, carga inicial, inicialização do servidor e execução dos jobs. É o ponto de montagem e operação da aplicação.
`shared/`	Construi todas as aplicações que são compartilhadas no servidor entre os modulos, como logger, sessão de banco de dados e autenticação JWT.

Regras de negócio

Disponibilidade do técnico

Situação informada no orçamento	Efeito no prazo
Técnico disponível	Mantém a duração base.
Técnico indisponível	Acrescenta **24 horas e 1 dia**, sem consultar a agenda.

Cálculo do prazo final

Prazo final = duração base + acréscimo do técnico + reposição das peças com estoque insuficiente.

Exemplo	Duração base	Acréscimos	Prazo final
Técnico disponível e estoque suficiente	48 horas / 2 dias	Nenhum	**48 horas / 2 dias**
Técnico indisponível e estoque suficiente	48 horas / 2 dias	Técnico: 24 horas / 1 dia	**72 horas / 3 dias**
Orçamento exige **4 velas**, mas há **2 em estoque**; técnico disponível	24 horas / 1 dia	Reposição das 2 velas faltantes: 48 horas / 2 dias	**72 horas / 3 dias**
Orçamento exige **1 bomba d'água**, mas o estoque está **zerado**; técnico disponível	24 horas / 1 dia	Reposição: 48 horas / 2 dias	**72 horas / 3 dias**
Técnico indisponível e duas peças distintas com estoque insuficiente	48 horas / 2 dias	Técnico: 24 horas / 1 dia; peça A: 24 horas / 1 dia; peça C: 48 horas / 2 dias	**144 horas / 6 dias**

Regras do cálculo

• Quantidades repetidas da mesma peça são somadas antes da comparação com o estoque.

• Cada peça distinta com saldo insuficiente acrescenta seu prazo de reposição **uma única vez**, independentemente da quantidade faltante. Os prazos das peças distintas são somados.

• Estoque insuficiente ou zerado permite criar o orçamento com prazo adicional. **Peça não cadastrada interrompe a criação.**

• Criar o orçamento não compra, reserva ou retira peças do estoque.

• Horas e dias são calculados separadamente, sem conversão automática nem calendário de dias úteis.